MetMemo

ARCHITECTURE DOCUMENT · V2.0

MetMemo System Architecture

A digital business card with an AI assistant that remembers everyone you meet, and tells you who to follow up with. This document describes how the system is built, how data flows through it, and why it was designed this way.

VERSION	DATE	STATUS
2.0 · as deployed	1 November 2026	Live in production
LIVE URL	**COURSE**	**REPLACES**
metmemo.app	Generative AI & Agentic AI capstone	v1.0, proposal (Oct 2026)

Contents

01	Overview	Purpose, scope, key facts
02	User workflow	From first card to sent follow-up
03	System architecture	Components and data flow
04	Laravel application	Layers and responsibilities
05	Follow-up agent	Tools, loop, one request end to end
06	Card scanner	Vision to structured, reviewable fields
07	Data model	Tables, vectors, ownership
08	Security, privacy and access control	Guardrails, roles, limits
09	Caching and observability	Redis, Langfuse, logs
10	Deployment	Production topology and operations
11	Evaluation	Method, results, iteration
12	Design decisions	What was chosen, and the trade-off
13	Failure handling	What happens when things go wrong
A	Repository map	Where each part lives in the code

Document history

VERSION	DATE	CHANGE
1.0	October 2026	Proposal: planned architecture; assistant, guardrails, evaluation and deployment marked "to build".
2.0	1 November 2026	As built and deployed: UUID keys, agent with five tools, guardrails, caching, tracing, evaluation results, role-based access control, production deployment.

01 Overview

Salespeople, brokers and business owners meet new people every week. Each meeting ends with a swapped card and a promise, and most of those promises are forgotten. MetMemo keeps the card, the conversation and the follow-up date together, and adds an assistant that answers from that memory.

Met is the people you meet: your digital card, its QR code, the lead form on your public page, card swaps between users, and the paper cards you scan.

Memo is the memory of each meeting: notes, follow-up dates, a daily reminder, and the assistant that recalls them and drafts your follow-up.

2	5	15/15	0
services: Laravel app and FastAPI AI service	agent tools, at most 5 steps per question	agent tasks completed after evaluation	messages sent without the user

Scope

AREA	WHAT IT INCLUDES
Capture	Digital cards with public page, QR code and vCard; lead form; user-to-user card swap; AI card scanner; manual entry.
Memory	Contacts, meeting notes, follow-up dates, dashboard, insights, daily 08:00 reminder, CSV export.
Assistant	Agentic follow-up assistant with tool calling, retrieval over notes, saved chats and drafts.
Administration	Role-based access (Super Admin, Admin, Member), module permissions, usage overview with counts only.
Operations	Docker Compose deployment, HTTPS, tracing, evaluation suite.

Principle throughout: **the AI suggests and drafts; the user decides and sends.**

02 User workflow

One full loop, from creating a card to sending a follow-up. Steps with a green border involve the AI.

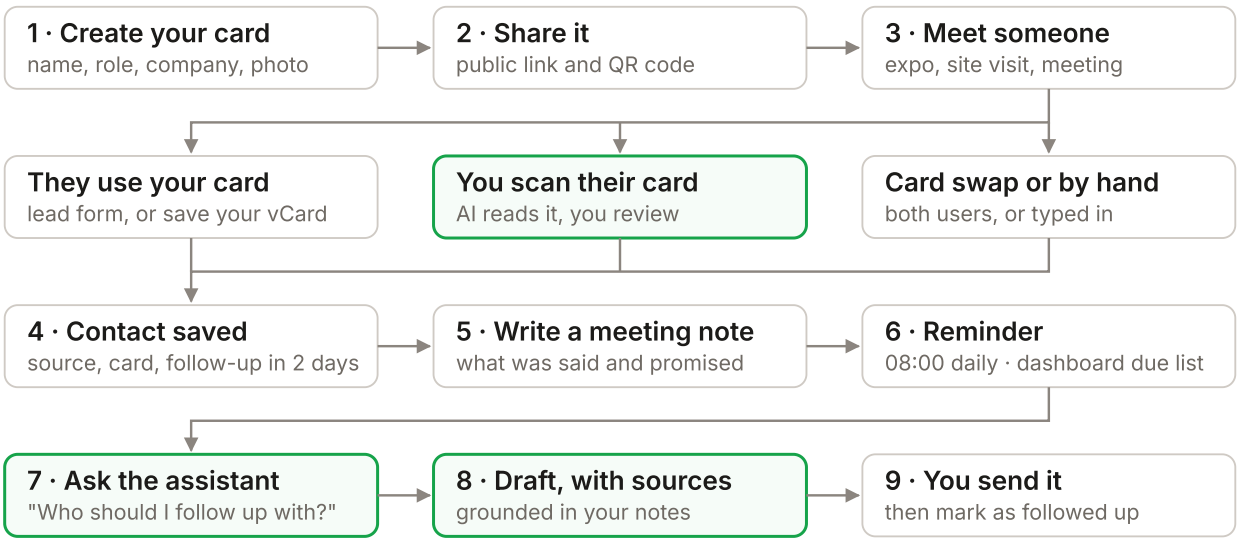


Figure 1. User workflow. Three capture paths lead to the same saved contact; the AI reads cards and drafts messages, but the user confirms and sends.

03 System architecture

Two applications. **Laravel** owns the users, the data and every page. The **Python AI service** owns every model call. Laravel never calls OpenAI directly: it calls the AI service over HTTP with a shared `X-Internal-Token`, and the assistant's tools read contacts back through Laravel's read-only `/internal` API with the same token. Ownership rules therefore live in one place, and the model never chooses whose data to read.

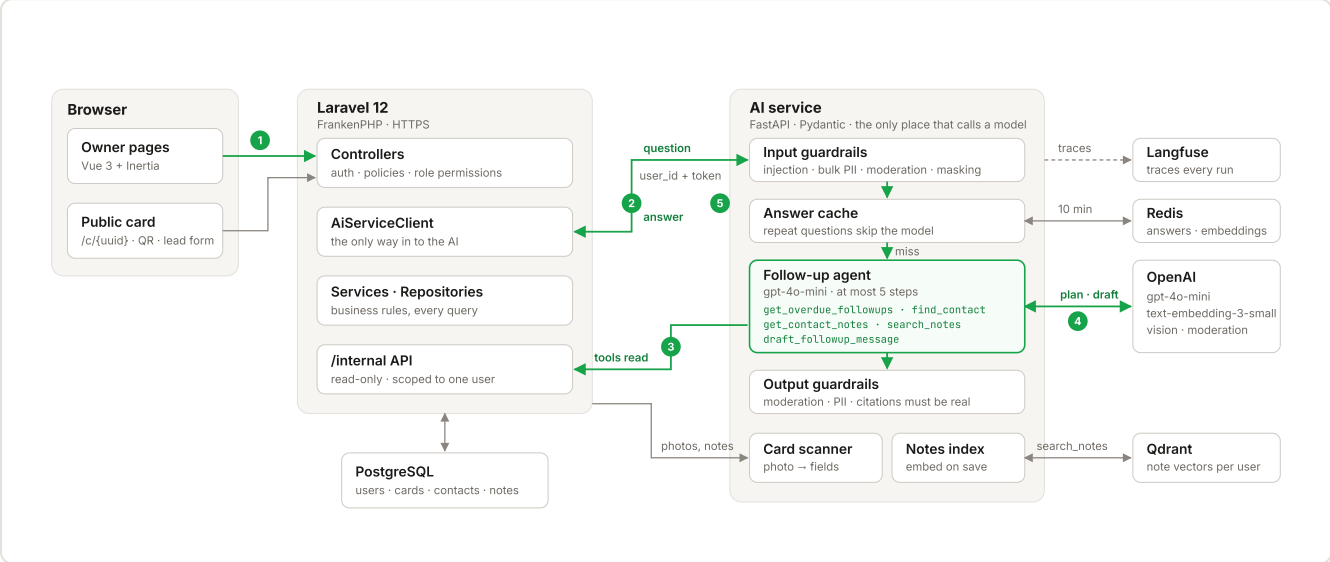


Figure 2. Components and data flow. The green path, numbered 1–5, is one assistant question; grey arrows are the scanner, note indexing, caching and tracing.

PART	TECHNOLOGY	RESPONSIBILITY
Web app	Laravel 12, Jetstream, Inertia, Vue 3, Tailwind	Login, 2FA, Google sign-in, cards, contacts, notes, dashboard, insights, export, public card, admin.
Database	PostgreSQL 17	All app data with UUID keys; every card, contact and chat belongs to one user.
AI service	FastAPI, Pydantic	The only place that calls a model; validates every request, tool argument and response.
LLM	OpenAI `gpt-4o-mini`, temperature 0	Plans, chooses tools, reads card photos, drafts messages.
Embeddings	`text-embedding-3-small`, 1536 dims	Turns `"{contact name}: {note}"` into a vector.
Vector store	Qdrant 1.19	Finds the notes that match a question, always filtered by `user_id`.
Cache	Redis 7	Embeddings and repeated answers.
Tracing	Langfuse	Tool calls, retrieved notes, latency, tokens and evaluation scores per request.

04 Laravel application

Every request passes through the same layers. Each layer has one job, which keeps the code testable.

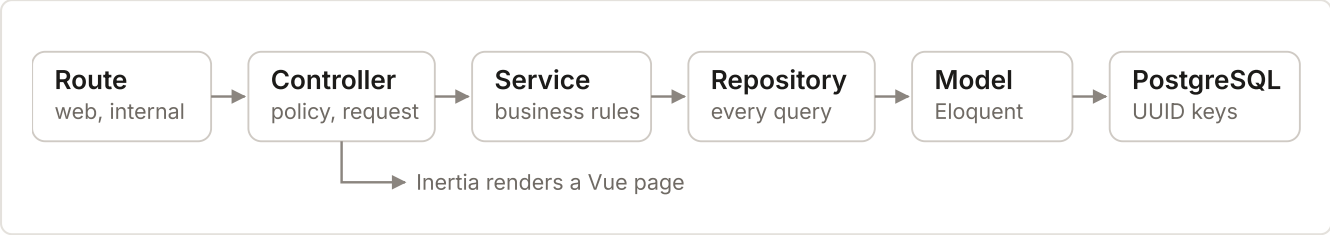


Figure 3. Request path through the Laravel application.

LAYER	FOLDER	EXAMPLES
Controller	`app/Http/Controllers`	Card, Contact, ContactNote, Scan, Assistant, Dashboard, Insights, ContactExport, PublicCard, `Admin/*`, `Internal/*`
Policy	`app/Policies`	Ownership: a user can only touch their own records.
Service	`app/Services`	CardService, ContactService, VCardService, ContactExportService, InsightsService, AdminStatsService, `Ai/AiServiceClient`, `Ai/NotesIndexer`
Repository	`app/Repositories`	Interfaces in `Contracts`, Eloquent implementations bound in `RepositoryServiceProvider`.
Access	`app/Support/Access.php`	Roles and module permissions (Spatie laravel-permission); see section 08.
Pages	`resources/js/Pages`	Dashboard, Cards, Contacts, Scan, Assistant, Insights, Admin, Public, Profile, Welcome.

05 Follow-up agent

The assistant does not answer in one shot. The model plans, calls tools for real data, reads the results and repeats until it can answer. The loop is bounded to **five model calls**; on the last step the tools are switched off, so the model must answer with what it has gathered.

TOOL	READS	PURPOSE
`get_overdue_followups`	`/internal/…/followups`	Who is overdue, due today and upcoming.
`find_contact`	`/internal/…/contacts`	Resolve a name, company or event to contacts; matches each word.
`get_contact_notes`	`/internal/…/notes`	One contact's notes (up to 20), masked and labelled as data.
`search_notes`	Qdrant	Semantic search over the user's own notes (RAG).
`draft_followup_message`	Notes already read	A draft that may cite only that contact's notes; never sent.

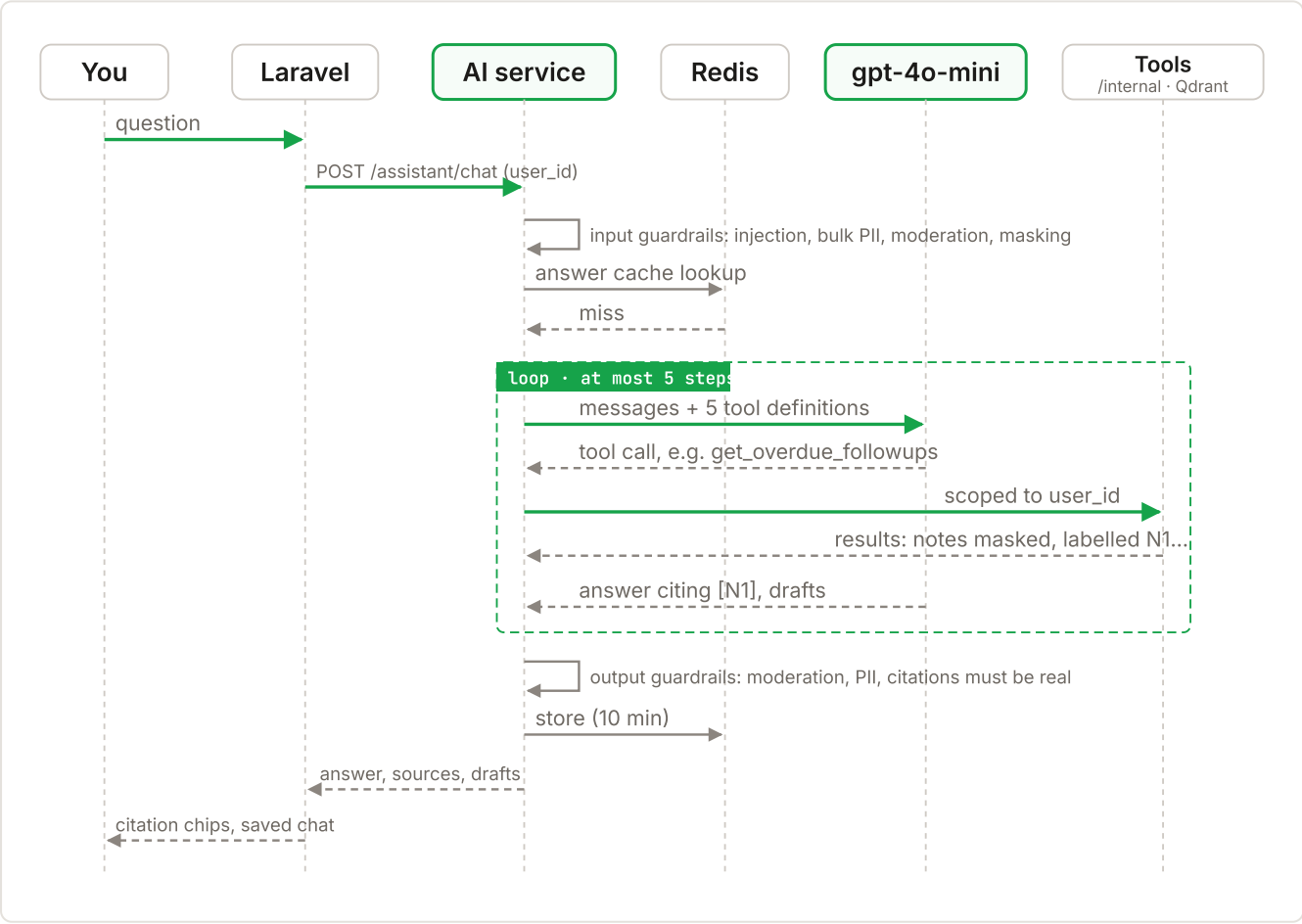


Figure 4. One assistant request. Solid arrows are calls, dashed arrows are replies. A refused input stops after the input guardrails, with zero model calls.

CAPABILITY	HOW METMEMO COVERS IT
Tool calling	Five tools with Pydantic-validated arguments.
Multi-step planning	Find who is due, read their notes, then draft, in one request.
RAG	`search_notes` over Qdrant, filtered to the user.
Memory	Chats are saved; the history is sent with each message.
Prompt variants	`baseline`, `structured`, `grounded` compared on the same cases; `grounded` kept (fewest steps, 2.1 on average).
Human in the loop	Drafts are marked "Not sent"; the user copies and sends them.

06 Card scanner

A photo of a paper card goes to `POST /scan` on the AI service. `gpt-4o-mini` with vision returns strict JSON for five fields: `full_name`, `email`, `phone`, `company`, `job_title`, each with a confidence score and a `needs_review` flag.

RULE	BEHAVIOUR
Never guess	Text is copied as printed; a missing field is null with confidence 0.
Flag doubt	Any field below 0.75 confidence, or flagged by the model, is highlighted as "Check this".
Human confirms	Nothing is saved until the user reviews the form and taps Save; the photo itself is never stored.
Injection in images	Text on the card that looks like instructions is ignored; only fields are extracted.
Rate limit	10 scans per minute per user.

07 Data model

Every card, contact and chat belongs to exactly one user, which is how privacy is enforced. Keys are UUIDs, so public card links (`/c/{uuid}`) cannot be guessed.

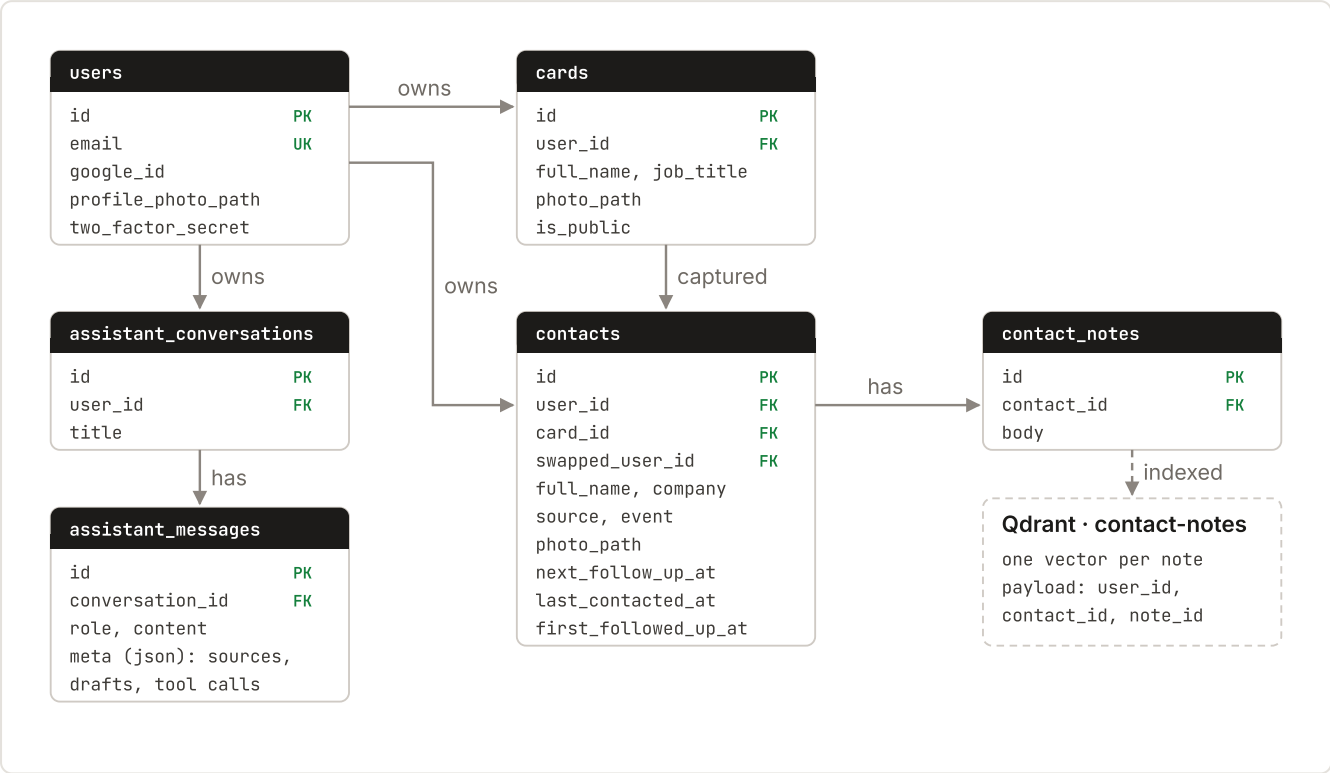


Figure 5. Core tables. Arrows point from one to many. All tables have `created_at` and `updated_at`.

TABLE	NOTES
contacts	`source` is manual, lead_form, scan or swap. Index on (`user_id` , `next_follow_up_at`) serves the due list.
contact_notes	The text the assistant reads; embedded as `"{contact name}: {note}"` when saved.
roles, permissions	Spatie laravel-permission with UUID keys; `permissions.module` groups permissions for the admin matrix.
notifications	Follow-up reminders created by the daily 08:00 job.
Framework	sessions, cache, jobs, personal_access_tokens, passkeys.

08 Security, privacy and access control

Guardrails

WHERE	CHECK	WHEN IT FIRES
Input	Prompt-injection patterns	Refused before any model call; local and free.
Input	Bulk data requests	"List every contact's phone number" is refused.
Input	OpenAI moderation	Refused, with escalation to MetMemo support.
Input	Phone numbers and emails	Masked to `[PHONE]` / `[EMAIL]` before the model sees them.
Tool results	PII and injected instructions in notes	Masked, and labelled for the model as data not to follow.
Output	Moderation, PII, grounding	Unsafe answers replaced; invented citations removed; ungrounded drafts rejected.
Design	Fixed `user_id`; drafts never sent	An injection cannot reach another user's data or send anything.

Roles and permissions

Every app module is a permission, grouped by module, so any role can be granted access. New users get **Member**. Routes check permissions server-side; the UI only hides what a user cannot use.

ROLE	PERMISSIONS	NOTES
Super Admin	All permissions	Built in and locked; only a Super Admin can grant it.
Admin	All except creating, editing and deleting roles	Built in.
Member	Workspace: cards, contacts, export, scan, assistant, insights	Default for every new account.
Custom roles	Any combination, by module	Created in Admin › Roles; deleting one moves its users back to Member.

> **Privacy guarantee.** The Super Admin shortcut applies to permission names only. Ownership policies still apply to everyone, so even a Super Admin cannot open another user's contacts, notes or chats. The admin overview shows counts only.

Other controls

CONTROL	DETAIL
Rate limits	Assistant 20/min, scanner 10/min, lead form 5/min, card swap 10/min, sign-in with Google 20/min.
Authentication	Jetstream with two-factor authentication, passkeys and Google sign-in.
Internal API	`/internal` requires the shared token; scoped bindings make another user's contact a 404.
Secrets	Only in `.env.production` files on the server (mode 600, git-ignored); none in the repository.
Logs and traces	PII is masked before logging and before traces are sent to Langfuse.

09 Caching and observability

CACHE	LIFETIME	INVALIDATION
Embeddings	30 days	Content-addressed: a changed note gets a new key; the TTL only bounds memory.
Answers	10 minutes	Any note change makes earlier answers stop matching; a new day starts fresh.

Redis holds nothing that cannot be rebuilt; losing it only costs speed. **Langfuse** records every request as a trace with its tool calls, retrieved notes, model calls, latency and tokens; evaluation runs are tagged by run and case so baseline and improved runs can be compared side by side.

10 Deployment

Production runs on one DigitalOcean droplet (Ubuntu 24.04, 2 GB RAM, Singapore) with Docker Compose. FrankenPHP's built-in Caddy obtains and renews the Let's Encrypt certificates. Only ports 22, 80 and 443 are open; PostgreSQL, Redis, Qdrant and the AI service have no public ports.

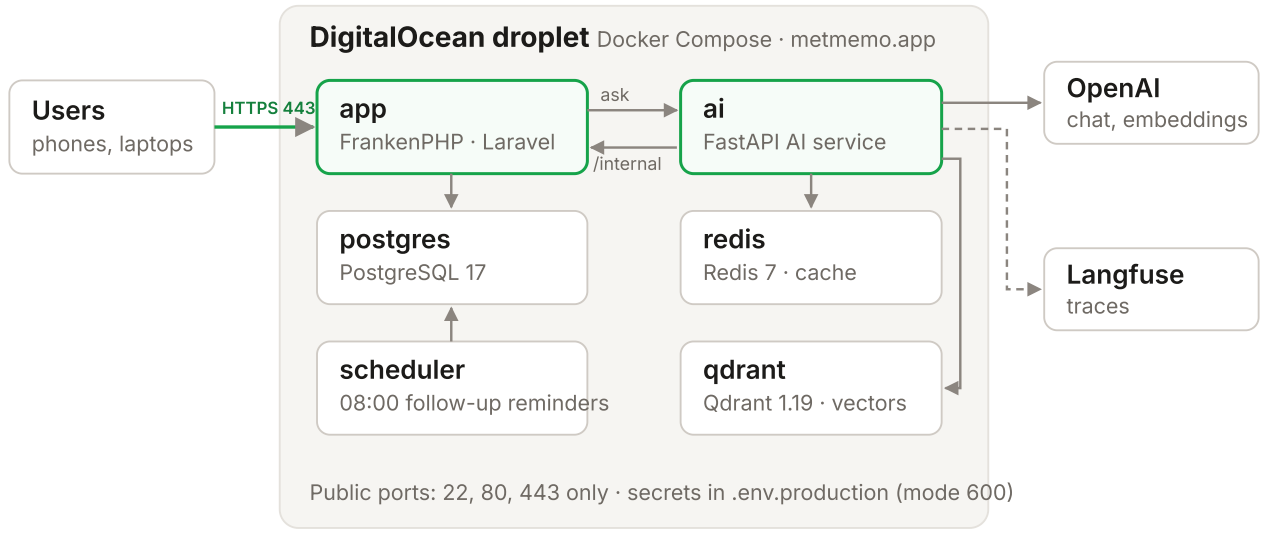


Figure 6. Production topology. Green boxes are the two application services.

TASK	COMMAND (FROM `/OPT/METMEMO`)
First setup	`deploy/setup-env.sh metmemo.app` generates the app key, database password and internal token.
Deploy or update	`deploy/deploy.sh`
Seed admin and demo data	`docker compose -f compose.prod.yml exec app php artisan db:seed --force`
Re-index notes	`… exec app php artisan metmemo:index-notes`
Logs	`docker compose -f compose.prod.yml logs -f ai app`

11 Evaluation

Model `gpt-4o-mini` at temperature 0 with the `grounded` prompt, on the seeded demo account (25 contacts, 42 notes). Each case runs the same steps as a real request, without the answer cache, and is traced in Langfuse.

SUITE	DATASET	METRICS
RAG	20 golden questions with reference answers; 13 name a contact, 7 describe a topic.	RAGAS faithfulness, answer relevancy, context precision and recall; retrieval recall@5 and MRR.
Agent	15 scenarios: 7 normal tasks, 8 failure cases (ambiguous name, no notes, unknown contact, off-topic, injection in a note, direct injection, third-party PII, mid-conversation).	Tool-selection accuracy, task completion.

Results

METRIC	BASELINE	IMPROVED
Task completion	13/15	**15/15**
Failure cases handled	7/8	**8/8**
Tool-selection accuracy	14/15	**15/15**
Retrieval recall@5, named questions	0.89	**1.00**
Retrieval MRR	0.91	**1.00**
Faithfulness	0.90	0.91
Context precision	0.85	0.88
Context recall	0.95	0.95
Answer relevancy	0.70	0.67

The iteration

FAILURE	ROOT CAUSE	FIX
F1 · Elena's note not retrieved	Only the note text was embedded; it never names Elena.	Embed `"{contact name}: {note}"` and re-index.
F7 · Off-topic answered	No rule said what is out of scope.	Scope rule in the `grounded` prompt.
F8 · "Raffy Mendoza" not found	One `LIKE` over the whole phrase.	Match each word, case-insensitively.

> **Regression caught.** The first scope rule made the assistant refuse real client questions, and context recall fell from 0.95 to 0.82. Rewording the rule to say what *is* in scope removed the regression, which is why the full suite runs after every change.

Limitations: small datasets (enough to catch failures, not to estimate rates precisely); the same model answers and judges; single runs at temperature 0.

12 Design decisions

DECISION	WHY	TRADE-OFF
Separate Python AI service	One place for model calls, keys, guardrails and tracing; the Python AI ecosystem.	A network hop and a second service to deploy.
Tools read through Laravel `/internal`	Ownership rules stay in one place; `user_id` is fixed per request, so the model cannot widen its scope.	An HTTP call per tool (10 s timeout).
`gpt-4o-mini`, **temperature 0**	Low cost and latency; deterministic for evaluation.	Less capable than larger models on long reasoning.
`grounded` **prompt**	Best of three variants on the same cases; fewest steps (2.1 on average).	The longest prompt of the three.
Bounded agent loop	At most 5 model calls; tools off on the last step. Predictable cost and latency.	Very broad questions may need narrowing.
Embed the contact's name with each note	Questions name people; notes often do not (fixes F1).	Notes must be re-indexed when the format changes.
Drafts are never sent	Human in the loop; an injection cannot send anything.	The user does the final step.
UUID keys	Unguessable public links and IDs.	Longer URLs.
Permissions per module (Spatie)	Any role can be granted any module; admin features use the same mechanism.	More checks to keep in sync with routes.
Single droplet, Docker Compose	Simple, low cost, automatic HTTPS.	One machine: no horizontal scaling or failover.

13 Failure handling

SITUATION	BEHAVIOUR
AI service down	Cards, contacts, notes, reminders and exports keep working; the assistant says it is unavailable and offers a retry.

SITUATION	BEHAVIOUR
No notes, or unknown contact	The assistant says so and gives only what it has, instead of inventing a conversation.
Tool error	Reported plainly to the user; nothing is guessed.
Step limit reached	"I couldn't finish that within my step limit. Try asking about one contact or one thing at a time."
Moderation service unavailable	Fails open and is logged (`input.moderation_unavailable`); the local pattern checks never depend on the network.
Too many requests	HTTP 429 with a plain message to wait a moment.
Scanner unsure or not configured	Unsure fields are flagged for review; without a key the scanner returns 503 and manual entry still works.

A Repository map

PATH	CONTENTS
`app/`	Laravel controllers, policies, services, repositories, models, access rules.
`resources/js/Pages`	Vue 3 pages rendered with Inertia.
`routes/web.php` , `internal.php`	App routes with permission middleware; the token-protected internal API.
`ai-service/app/agent`	Agent loop, tools, prompts, model client.
`ai-service/app/guardrails.py`	Input, note and output guardrails.
`ai-service/app/card_reader.py`	Card scanner.
`ai-service/evals`	Golden set, scenarios and the evaluation runner.
`docs/evidence`	Evaluation, guardrails, caching, prompt decision and failure log.
`deploy/` , `compose.prod.yml`	Production setup and deployment scripts.
`tests/` , `ai-service/tests`	Feature tests for Laravel and the AI service.